

Lecture No-2: Linux Commands and Process Management

Duration: 1 Hour

Objective: Learn essential Linux commands and process scheduling in HPC environments.

Topics Covered:

- Linux Commands: File navigation (ls, cd, pwd), file manipulation (cp, mv, rm), and system monitoring (top, ps).
- Process Scheduling: Process states, scheduling algorithms (e.g., Round Robin, Priority-based).
- Process Synchronization: Concepts of semaphores, locks, and their relevance in HPC.

Introduction

Welcome to Lecture No-2 on Linux Commands and Process Management. This lecture introduces fundamental Linux commands for navigating and managing files, crucial for working in High-Performance Computing (HPC) systems. We will also explore how the operating system schedules processes and the synchronization techniques that ensure efficient parallel execution in HPC environments. By the end, you will have a solid understanding of these concepts and their relevance to HPC.

Linux Commands

Linux commands are the primary interface for interacting with the operating system, enabling tasks from file operations to system monitoring. These are especially important in HPC for managing resources and workflows.

File Navigation

- **ls**
 - **Purpose:** Lists directory contents.
 - **Syntax:** ls [options] [directory]
 - **Example:** ls -l /home displays a detailed list of files in the /home directory, including permissions, ownership, and size.

- **Options:** -l (long format), -a (all files, including hidden), -h (human-readable sizes).
-
- **cd**
 - **Purpose:** Changes the current working directory.
 - **Syntax:** cd [directory]
 - **Example:** cd /var/log navigates to the /var/log directory, commonly used for system logs.
 - **Tips:** Use cd ~ for home directory, cd .. for parent directory.
- **pwd**
 - **Purpose:** Prints the current working directory.
 - **Syntax:** pwd
 - **Example:** /home/user might be output, showing your location in the file system.

File Manipulation

- **cp**
 - **Purpose:** Copies files or directories.
 - **Syntax:** cp [options] source destination
 - **Example:** cp file1.txt file2.txt creates a copy of file1.txt.
 - **Options:** -r (recursive) for directories, e.g., cp -r dir1 dir2.
- **mv**
 - **Purpose:** Moves or renames files/directories.
 - **Syntax:** mv [options] source destination
 - **Example:** mv oldname.txt newname.txt renames the file.
 - **Use Case:** Move files with mv file.txt /path/to/directory.
- **rm**

- **Purpose:** Removes files or directories.
- **Syntax:** `rm [options] target`
- **Example:** `rm -r directory` deletes a directory and its contents.
- **Caution:** Use `-rf` (recursive, force) carefully to avoid unintended data loss.

System Monitoring

- **top**
 - **Purpose:** Displays real-time system process information.
 - **Syntax:** `top`
 - **Example:** Shows CPU/memory usage, process IDs; press `q` to quit.
 - **Use Case:** Sort by CPU (P) or memory (M) to monitor resource usage.
- **ps**
 - **Purpose:** Provides a snapshot of current processes.
 - **Syntax:** `ps [options]`
 - **Example:** `ps aux` lists all processes with details like user, PID, and resource usage.
 - **Options:** `aux` (all users, detailed view).

Process Scheduling

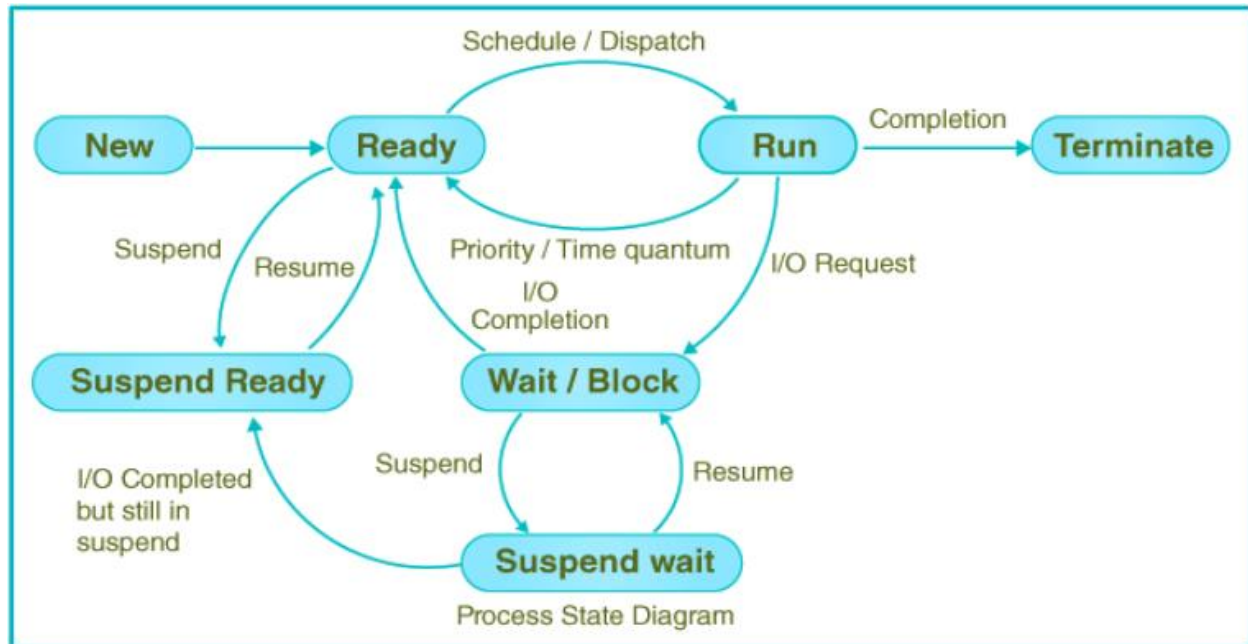
A process is a running instance of a program. In HPC, efficient process scheduling ensures optimal resource utilization across compute nodes.

Process States

Processes transition through states during execution:

- **New:** Process is being created.
- **Ready:** Waiting for CPU allocation.

- **Running:** Executing on the CPU.
- **Waiting:** Awaiting an event (e.g., I/O).
- **Terminated:** Process has completed.



Scheduling Algorithms

The operating system uses algorithms to manage process execution:

- **Round Robin:**
 - Each process gets a fixed time slice (quantum).
 - Fair, but may increase context-switching overhead.
 - **Example:** Ideal for time-sharing systems.

Explanation of Round Robin Scheduling Algorithm

Round Robin (RR) is a widely used CPU scheduling algorithm designed to ensure fairness in time-sharing systems. In this algorithm, each process in the system is assigned a fixed time slice, known as a **time quantum**, during which it can execute on the CPU. Once the time quantum expires, if the process hasn't completed, it is **preempted** (interrupted) and moved to the end of the **ready queue**, allowing the next process in line to take its turn. This cyclic execution continues until all processes are completed. The key features of Round Robin are:

- **Preemptive:** Processes can be interrupted after their time quantum ends.
- **Fairness:** Every process gets an equal opportunity to use the CPU.
- **Responsiveness:** Suitable for systems where multiple users or tasks need CPU time without long delays.

The performance of Round Robin depends heavily on the size of the time quantum:

- A **small time quantum** may lead to frequent context switching, increasing overhead.
- A **large time quantum** may reduce responsiveness, resembling First-Come-First-Served (FCFS) scheduling.

Simple Example

Let's illustrate how Round Robin works with a simple example. Suppose we have **three processes** with the following **burst times** (the total CPU time each process needs to complete):

P1: 10 units

P2: 5 units

P3: 8 units

Assume the **time quantum** is **4 units**. All processes arrive at time 0 and are placed in the ready queue in the order **P1, P2, P3**. Here's how the scheduling unfolds step by step:

Execution Timeline

Time 0–4: P1 runs for 4 units.

Remaining time for P1: $10 - 4 = 6$ units.

Queue after this: P2, P3, P1.

Time 4–8: P2 runs for 4 units.

Remaining time for P2: $5 - 4 = 1$ unit.

Queue after this: P3, P1, P2.

Time 8–12: P3 runs for 4 units.

Remaining time for P3: $8 - 4 = 4$ units.

Queue after this: P1, P2, P3.

Time 12–16: P1 runs for 4 units.

Remaining time for P1: $6 - 4 = 2$ units.

Queue after this: P2, P3, P1.

Time 16–17: P2 runs for 1 unit (it only needs 1 unit to finish).

P2 completes (remaining time = 0).

Queue after this: P3, P1.

Time 17–21: P3 runs for 4 units.

P3 completes (remaining time = 0).

Queue after this: P1.

Time 21–23: P1 runs for 2 units.

P1 completes (remaining time = 0).

Queue is now empty.

Final Schedule

The execution sequence can be represented as:

Time	0–4	4–8	8–12	12–16	16–17	17–21	21–23
Process	P1	P2	P3	P1	P2	P3	P1

Total time: 23 units (when all processes complete).

Completion order: P2 (at 17), P3 (at 21), P1 (at 23).

Key Observations

P1 runs three times: 4 units, 4 units, then 2 units (total 10 units).

P2 runs twice: 4 units, then 1 unit (total 5 units).

P3 runs twice: 4 units, then 4 units (total 8 units).

If a process has less time remaining than the time quantum (e.g., P2's 1 unit or P1's 2 units), it runs only for the remaining time and completes.

This example demonstrates how Round Robin ensures that no single process monopolizes the CPU, promoting fairness and allowing all processes to progress toward completion in a predictable, turn-based manner.

- **Priority-based:**

- Processes are prioritized; higher-priority ones run first.
- Priorities can be static or dynamic.
- **Example:** In HPC, critical jobs may get higher priority.

Explanation of Priority-Based Scheduling with a Simple Example

Priority-based scheduling is a method used by operating systems or computing environments to determine the order in which processes are executed. In this approach, each process is assigned a **priority**, and the scheduler ensures that **higher-priority processes run before lower-priority ones**. If multiple processes have the same priority, they might be scheduled using another rule, like first-come, first-served. Priorities can be:

- **Static:** Fixed for the duration of the process.
- **Dynamic:** Adjustable during execution (e.g., to prevent a low-priority process from waiting too long).

In **High-Performance Computing (HPC)**, priority-based scheduling is often used to ensure that **critical jobs**—such as time-sensitive simulations—get the resources they need quickly by assigning them higher priorities over less urgent tasks.

To illustrate how this works, let's walk through a **simple example** using **preemptive priority scheduling**, where a higher-priority process can interrupt (or "preempt") a lower-priority process that's currently running.

Simple Example: Preemptive Priority Scheduling

Imagine we have **three processes** with different priorities, arrival times, and execution times. In this example, we'll assume that a **lower priority number means higher priority** (e.g., priority 1 is the highest, priority 3 is the lowest). Here are the details:

- **P1**: Priority = 1 (highest), Burst time = 5 units, Arrival time = 4
- **P2**: Priority = 2, Burst time = 5 units, Arrival time = 0
- **P3**: Priority = 3 (lowest), Burst time = 5 units, Arrival time = 2

Burst time is the amount of time a process needs to complete its execution. Since this is **preemptive scheduling**, a process can be interrupted if a higher-priority process arrives.

Step-by-Step Execution

1. **Time 0:**
 - **P2** arrives (priority 2) and starts running because no other processes are present.
2. **Time 2:**
 - **P3** arrives (priority 3). Since **P2** has a higher priority ($2 < 3$), **P2** continues running.
3. **Time 4:**
 - **P1** arrives with priority 1 (highest). Because **P1** has a higher priority than **P2** ($1 < 2$), it **preempts** (interrupts) **P2**.
 - At this point, **P2** has run for 4 units (from time 0 to 4) and still needs 1 unit to finish.
 - **P1** starts running.
4. **Time 4 to 9:**
 - **P1** runs for its full burst time of 5 units and completes at time 9.
5. **Time 9:**
 - After **P1** finishes, the scheduler picks the next highest-priority process, which is **P2** (priority 2).
 - **P2** resumes from where it left off, needing only 1 more unit. It runs from time 9 to 10 and finishes.

6. Time 10:

- Now, only **P3** (priority 3) remains. It starts running and takes 5 units, finishing at time 15.

Execution Timeline

Here's how the processes execute over time:

Time Range 0–4 4–9 9–10 10–15

Process P2 P1 P2 P3

- P2** runs from time 0 to 4 (4 units), gets preempted by **P1**, and later resumes from 9 to 10 (1 unit).
- P1** runs from time 4 to 9 (5 units) after interrupting **P2**.
- P3** runs from time 10 to 15 (5 units) once **P2** is done.

Key Observations

- P1**, with the highest priority (1), interrupts **P2** when it arrives at time 4, showing how higher-priority processes take precedence.
- P2** is split into two parts (0–4 and 9–10) due to preemption.
- P3**, with the lowest priority (3), waits until both **P1** and **P2** finish.

Connecting to HPC

In an **HPC environment**, this example mirrors how critical jobs are prioritized. For instance, if **P1** represents a time-sensitive scientific simulation (e.g., weather forecasting), it would be assigned a high priority (like 1) to ensure it runs as soon as possible, even if it means interrupting less urgent tasks like **P2** (e.g., a routine data analysis).

Static vs. Dynamic Priorities

In this example, we used **static priorities**, where each process keeps its assigned priority (1, 2, or 3) throughout its execution. In a **dynamic priority system**, priorities might change—for example, **P3**'s priority could increase over time if it waits too long, preventing it from being delayed indefinitely (a problem called starvation).

This example demonstrates how priority-based scheduling ensures that higher-priority processes run first, making it a powerful tool for managing tasks in systems like HPC where certain jobs demand immediate attention.

HPC Relevance: Scheduling in HPC often involves job queues managed by systems like SLURM, optimizing for throughput and resource allocation.

Process Synchronization

In HPC, multiple processes may run concurrently, sharing resources. Synchronization prevents conflicts (e.g., race conditions) and ensures data consistency.

Why Synchronization?

Parallel processes accessing shared data need coordination to avoid errors. For example, two processes updating a counter simultaneously could produce incorrect results without synchronization.

Semaphores

- **Definition:** Variables controlling resource access.
- **Types:**
 - **Binary Semaphore:** Acts as a lock (0 or 1).
 - **Counting Semaphore:** Allows multiple accesses up to a limit.
- **Example:** A binary semaphore ensures only one process updates a shared variable.

Locks

- **Definition:** Mechanisms to restrict resource access.
- **Types:**
 - **Spinlocks:** Process waits actively, good for short delays.
 - **Mutexes:** Process sleeps, efficient for longer waits.
- **Example:** In HPC, mutexes protect shared data in parallel programs.

HPC Relevance: Synchronization is critical in parallel computing, but overhead must be minimized to maintain performance.

Summary

- **Linux Commands:** Mastered file navigation (ls, cd, pwd), manipulation (cp, mv, rm), and monitoring (top, ps) for HPC workflows.
 - **Process Scheduling:** Learned process states and algorithms (Round Robin, Priority-based), vital for HPC resource management.
 - **Process Synchronization:** Understood semaphores and locks, ensuring correct parallel execution in HPC.
-

Further Reading

- **Linux Commands:** man pages (e.g., man ls, man top).
- **Process Management:** Operating system textbooks.
- **HPC Context:** Online tutorials on HPC tools and techniques.